

Introduction to Electronic Design Automation

National Taiwan University

Spring 2026

Problem Set 1

Due by 2026/4/17 23:59.

1 [Design Methodology and Design Flow]

(12%)

- (a) (4%) *Modeling, design, and analysis* are three key processes in model-based design. Consider the design of an implantable electronic device for seizure detection and prevention in the brain. Elaborate on the key aspects that must be modeled, designed, and analyzed to accomplish this task.
- (b) (4%) Hardware/software co-design is an effective methodology for system implementation, allowing performance, latency, energy, and flexibility trade-offs. Consider implementing a real-time image classification system based on a Convolutional Neural Network (CNN) to process a continuous stream of images from a camera and output predicted labels. Let the system requirements be
- Input: 224×224 RGB images
 - Throughput: ≥ 15 frames per second
 - Platform: Embedded system (CPU + optional FPGA/ASIC accelerator)
 - Power budget: Limited (edge device scenario)

Assume the system consists of the following pipeline components: 1. image pre-processing, convolutional layers, activation functions (ReLU), pooling layers, fully connected layers, output (softmax+classification). Decide which components should be implemented in hardware (e.g., FPGA/ASIC accelerator) and which in software (CPU) and justify your answer.

- (c) (4%) What are the main synthesis tasks in VLSI design flow that converts system specification into physical circuit layout? In each of the tasks, how a design is abstracted/represented?

2 [Algorithm and Complexity]

(12%) Given a sequence of real numbers $a_n, a_{n-1}, \dots, a_1, a_0$, and a real number x , we would like to compute the value of the polynomial

$$P_n(x) = a_n x^n + a_{n-1} x^{n-1} + \dots + a_1 x + a_0.$$

- (a) (6%) Analyze the computational complexity of the recursive computation

$$P_n(x) = P_{n-1}(x) + a_n x^n,$$

where $P_{n-1}(x) = P_{n-2}(x) + a_{n-1}x^{n-1}$, ..., and $P_0(x) = a_0$. In particular, how many multiplications and additions are needed?

- (b) (6%) Analyze the computational complexity of the recursive computation

$$P_n(x) = x \cdot P'_{n-1}(x) + a_0,$$

where $P'_{n-1}(x) = x \cdot P'_{n-2}(x) + a_1, \dots$, and $P'_0 = a_n$. In particular, how many multiplications and additions are needed?

3 [NP-Completeness and Polynomial-Time Reduction]

(20%)

- (a) (10%) Prove that the feasibility problem of 0,1-ILP constraints (linear programming with variables taking on integer values 0 and 1) is NP-complete given that SAT is known to be NP-complete.
- (b) (10%) Prove that the Circuit-SAT problem is NP-complete given that 0,1-ILP is known to be NP-complete.

4 [SAT and Integer Linear Programming]

(16%) Consider k -coloring the graph $G(V, E)$ with $V = \{1, 2, 3, 4, 5, 6, 7\}$ and $E = \{(1, 2), (2, 3), (3, 4), (4, 5), (5, 1), (1, 6), (3, 6), (2, 7), (4, 7)\}$ such that no two vertices connected by an edge receive the same color.

- (a) (8%) Encode the above k -coloring instance, for $k = 2, 3$, as a SAT instance and use the solver `MiniSat` (<http://minisat.se/>) to solve. Show your input to and output from `MiniSat`.
- (b) (8%) Encode the above k -coloring instance, for $k = 2, 3$, as an ILP instance and use the solver `lp_solve` (<http://lpsolve.sourceforge.net/>) to solve. Show your input to and output from `lp_solve`.

5 [Model of Computation]

(20%)

- (a) (5%) Derive a *deterministic* finite automaton that accepts the regular expression

$$(01^*0)^*1,$$

with alphabet $\Sigma = \{0, 1\}$. (Hint: It could be converted from a nondeterministic finite automata.)

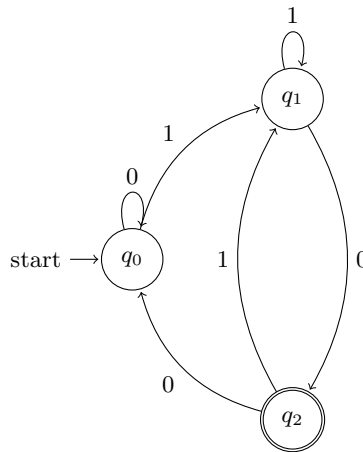


Fig. 1. A finite automaton.

- (b) (5%) Derive a regular expression that describes the language accepted by the automaton of Figure 1.
- (c) (5%) We are designing a timed access control system for a secure room. The system monitors user actions to ensure safety and proper sequencing. There is a button **a** to request entry and a button **b** to confirm entry after waiting the correct amount of time. Draw a timed automaton that describes the system behavior:
1. The system starts in the **Idle** state. No one is currently requesting entry.
 2. When a person presses **a**: The system moves to the **Requesting** state. A clock x starts to measure the waiting time for confirmation.
 3. While in **Requesting**, the person can: Press **a** again (repeat request) within 2 seconds, resetting the clock each time. Press **b** to confirm entry if at least 1 second has elapsed since the last **a**. Once **b** is pressed successfully, the system moves to state **Access Granted**.
 4. In **Access Granted**, the person is considered inside. The system returns to **Idle** automatically, ready for the next person to press **a**.
 5. Any additional person must wait until the system is back in **Idle** to start their own request.
- (d) (5%) A Petri net is a mathematical modeling tool used to describe and analyze systems with concurrent, asynchronous, or distributed processes. Model the following producer-consumer system with a bounded buffer using a Petri net, practicing synchronization and resource management:
1. Producers: There are two types of producers: High Priority (HP) and Low Priority (LP). Each producer generates items for the shared buffer. High-priority producers should be allowed to produce whenever possible, even if low-priority producers are waiting.
 2. Consumers: There are multiple consumers removing items from the buffer. Consumers always consume the oldest available item first (FIFO).

3. Buffer: Has a fixed capacity N . If the buffer is full, producers must wait.
If the buffer is empty, consumers must wait.
Assume we have 2 HP and 1 LP producers, 2 consumers, and a buffer of capacity $N = 3$.

6 [Scheduling]

(20%) Consider the following computation:

$$z = (a + b) \times (c + d) + (e \times f)$$

- (a) (4%) Draw the corresponding data-flow graph (DFG).
- (b) (4%) Perform ASAP scheduling and identify the required hardware resources in terms of multipliers and adders. Draw the time chart of the scheduling result.
- (c) (4%) Perform ALAP scheduling (assuming the entire DFG computation is to be finished with the shortest possible time span) and identify the required hardware resources. Draw the time chart of the scheduling result.
- (d) (4%) Perform the mobility-based scheduling and identify the required hardware resources. Draw the time chart of the scheduling result.
- (e) (4%) Given two arithmetic logic units (ALUs) A_1 and A_2 , let A_1 be capable of doing multiplication and addition, and let A_2 be capable of doing addition only. Perform critical-path list scheduling. Draw the time chart of the scheduling result.